DISHA

Developer Handbook

Architecture · Structure · Standards

Frontend (HOST) · Module Frontend · Backend API

v1.0 · 2026

01 Introduction

This handbook defines the engineering standards for MERN Stack development across all repositories — frontend HOST application, module frontends, and backend API services. It covers folder structure, naming conventions, architectural boundaries, and best practices to ensure consistency, scalability, and MFE-readiness across teams.

What this handbook covers

- Frontend (HOST) Folder Structure & Architecture
- Module Frontend (MFE) Folder Structure
- Backend (Express/Node) Folder Structure
- Naming Conventions across all layers
- Architectural Boundaries & Team Ownership

Layer	Technology	Purpose
Frontend HOST	React + TypeScript	Shell application, shared UI
Module Frontend	React + MFE	Feature modules, micro-frontends
Backend API	Node + Express	Domain-driven REST API
Database	MongoDB + Prisma	Data persistence layer

02 Frontend HOST — Folder Structure

The HOST application uses a feature-oriented, scalable structure designed to support Micro-Frontend (MFE) architectures. Each directory has a clear responsibility and boundary.

src/ — Root Structure

```
src/
├── assets/
├── config/
├── shared/
├── features/
├── hooks/
├── layouts/
├── pages/
├── services/
├── styles/
├── stores/
├── validators/
├── utils/
└── types/
```

2.1 assets/

Contains all static resources used by the application. No logic or cross-imports allowed.

assets/

```
assets/
├── images/
├── icons/
└── fonts/
```

Rules

- No logic or JavaScript
- No imports between asset files
- Organize by file type (images, icons, fonts)

2.2 config/

Holds global and environment-specific configuration including feature flags, federation setup, routing, and application constants.

config/

```
config/
├── env.ts          # Environment variables & validation
├── federation.ts   # Module Federation configuration
├── routes.ts       # App-level route definitions
```

```
└── constants.ts    # Global constants
```

Responsibilities

- Feature flags management
- Module Federation config
- App-level constants
- Environment variable access patterns

2.3 shared/

Houses reusable, dumb (presentational) components and styling utilities. Must remain free of business logic and API calls.

shared/

```
shared/
├── components/
│   ├── Button/
│   ├── Input/
│   └── Modal/
├── styles/
└── index.ts        # Public API barrel export
```

Rules

- No business logic
- No API calls or data fetching
- No feature-specific dependencies
- Export everything through index.ts

2.4 features/

The core business domain layer. Each feature is self-contained with its own components, hooks, services, validators, and types. Features are designed for easy extraction into standalone MFEs.

features/

```
features/
└── User/
    ├── components/      # Feature-specific UI
    ├── hooks/           # Feature-specific hooks
    ├── services/        # Feature API calls
    ├── validators/      # Feature validation schemas
    ├── types/           # Feature TypeScript types
    └── index.ts         # Public API — only export here
```

Feature Responsibilities

- Owns all business logic for the domain
- Exposes a clean public API via index.ts
- Internal implementation details stay private
- Can be extracted into an MFE with minimal refactoring

2.5 hooks/

Global reusable React hooks that are not tied to any specific feature. Place here only if a hook is consumed by more than one feature.

```
hooks/
hooks/
├── useDebounce.ts
├── useAuth.ts
└── useMediaQuery.ts
```

2.6 layouts/

Reusable page layout wrappers that define the structural shell — navigation, sidebars, content areas. Should not contain business logic.

```
layouts/
layouts/
├── AuthLayout.tsx       # Unauthenticated pages
└── DashboardLayout.tsx  # Authenticated app shell
```

2.7 pages/

Route-level components that compose features and shared components. Pages act as glue — they own routing context but no business logic.

```
pages/
pages/
├── Login.tsx
├── Profile.tsx
└── NotFound.tsx
```

Rule

- No business logic — pages are composition only
- No direct API calls from page files
- Import from features/ via their public index.ts

2.8 services/

Global API infrastructure including the HTTP client and interceptors. Feature-specific API calls live inside features/; only shared infrastructure lives here.

```
services/
services/
├── httpClient.ts       # Axios/Fetch instance config
└── authInterceptor.ts  # Token injection, refresh logic
```

2.9 styles/

Global styling configuration. Feature-scoped styles live inside features/; only app-wide tokens and configuration live here.

```
styles/
styles/
├── theme.ts            # Design token definitions
├── variables.css       # CSS custom properties
└── tailwind.config.ts  # Tailwind configuration
```

2.10 stores/

Global state management stores (e.g., Zustand, Redux Toolkit). Feature-scoped state lives inside features/; only truly global state lives here.

```
stores/
stores/
├── auth.store.ts   # Authentication state
└── user.store.ts   # Current user session
```

2.11 validators/

Global validation schemas using Zod or Yup. Shared schemas used across multiple features live here. Feature-specific schemas live inside the feature.

```
validators/
validators/
├── email.schema.ts
└── password.schema.ts
```

2.12 utils/

Pure helper functions with no side effects, no external dependencies, and no React context. These are safe to import anywhere.

```
utils/
utils/
```

```
├── formatDate.ts    # Date formatting utilities
└── debounce.ts      # Debounce implementation
```

2.13 types/

Global TypeScript type contracts shared across the application. These define API response shapes, shared data models, and utility types.

types/

```
types/
├── api.types.ts     # HTTP response contracts
└── user.types.ts    # User data models
```

03 Naming Conventions

Consistent naming ensures readability, predictability, and easy navigation — especially in large teams and micro-frontend environments. All engineers must follow these conventions strictly.

Type	Convention	Example
Folders	kebab-case	user-profile/, auth-flow/
.tsx Files (React)	PascalCase	UserProfile.tsx, AuthLayout.tsx
.ts Files (Non-React)	camelCase	httpClient.ts, formatDate.ts
Stores	<name>.store.ts	auth.store.ts, cart.store.ts
Validators	<name>.schema.ts	email.schema.ts, user.schema.ts
Hooks	use<Name>.ts	useAuth.ts, useDebounce.ts
Index files	index.ts	features/User/index.ts

3.1 Folder Naming

All folders use kebab-case. This represents a logical domain or grouping.

```
Folder Naming Examples
✅ user-profile/
✅ auth-flow/
✅ payment-methods/

❌ UserProfile/
❌ userProfile/
```

3.2 .tsx Files (React Components)

React component files use PascalCase. One component per file — no exceptions.

```
Component File Naming
✅ Login.tsx
✅ UserProfile.tsx
✅ AuthLayout.tsx
```

3.3 .ts Files (Non-React)

Non-component TypeScript files use camelCase and should describe the file's primary responsibility.

```
TypeScript File Naming
✅ httpClient.ts
✅ formatDate.ts
✅ useDebounce.ts
```

3.4 Stores

Store files use the pattern <name>.store.ts. This makes global state explicit, highlights side effects, and prevents accidental imports in pure layers.

Store File Naming

```
✅ auth.store.ts
✅ user.store.ts
✅ cart.store.ts
```

3.5 Validators / Schemas

Schema files use the pattern <name>.schema.ts. This clearly indicates validation logic and enforces architectural boundaries.

Schema File Naming

```
✅ email.schema.ts
✅ password.schema.ts
✅ user.schema.ts
```

3.6 Types

Type files use camelCase for grouping files, and PascalCase for exported interfaces and types within.

Type Definitions

```
// user.types.ts
export interface User {
  id: string
  name: string
  email: string
}

export type UserRole = "admin" | "user" | "guest"
```

3.7 Hooks

Hook files use the pattern use<Name>.ts. This follows React's convention and makes hooks instantly identifiable in the codebase.

Hook File Naming

```
✅ useAuth.ts
✅ useDebounce.ts
✅ useMediaQuery.ts
```

3.8 Index Files

Each feature and shared module exposes a public API via index.ts. This controls what is visible outside the folder and reduces deep import paths.

index.ts — Public API

```
// features/User/index.ts

// ✅ Public API — only export what consumers need
export { UserProfile } from "./components/UserProfile"
export { useUserData } from "./hooks/useUserData"
export type { User } from "./types/user.types"

// ❌ Internal implementation — never export
// export { userApiCalls } from "./services/user.service"
```

04 Module Frontend — Folder Structure

The Module Frontend repository uses a high-level application partitioning layer. It groups related pages and micro-frontends under logical modules, enabling independent team ownership and future MFE extraction.

src/ — Module Repo Root

```
src/
├── assets/
├── config/
├── shared/
├── modules/
├── services/
└── utils/
```

4.1 modules/

The modules/ directory is the primary partitioning layer. Each module represents a logical application domain — not a business feature. Modules can contain multiple pages and one or more micro-frontends.

modules/ — Structure

```
modules/
├── leave/
│   ├── pages/
│   │   ├── leaveSummaryPage.tsx
│   │   └── leaveApplicationPage.tsx
│   └── microFrontends/
│       └── leaveHistory/
│           ├── components/
│           ├── hooks/
│           ├── interfaces/
│           └── styles/
└── apar/
    ├── pages/
    └── microFrontends/
```

Module Purpose

- Split the application into independently owned functional modules
- Avoid flat pages/ directories as the app grows
- Prepare for Module Federation / MFE adoption
- Enable individual team ownership per module

4.2 pages/

Page components are scoped to their parent module. They compose features, layouts, and shared components without owning business logic.

```
module pages/
modules/leave/pages/
├── leaveSummaryPage.tsx      # Summary dashboard view
└── leaveApplicationPage.tsx  # New leave application form
```

Rules

- No direct API calls — delegate to services or hooks
- No state ownership — use stores or context
- No cross-module imports — modules must be isolated
- Route-level components only

4.3 microFrontends/

Each micro-frontend is an independently deployable runtime unit. It can be exposed or consumed via Module Federation and must communicate only through defined contracts.

```
microFrontends/
modules/leave/microFrontends/
└── leaveHistory/
    ├── components/     # MFE-internal UI components
    ├── hooks/          # MFE-internal hooks
    ├── interfaces/     # Public contracts (props/events)
    └── styles/         # Scoped styles
```

MFE Rules

- Treat each micro-frontend as a mini-application
- Owns its internal features, hooks, services, and state
- Communicates only via defined contracts (props/events/custom events)
- No direct imports from sibling MFEs or parent modules
- Fully isolated runtime boundary

05 Backend API — Folder Structure

The backend follows a domain-driven design aligned with the frontend's MFE architecture. Each module maps to an application domain, enabling clean separation, independent scaling, and clear ownership.

Backend src/ — Root Structure

```
src/
├── config/
├── modules/
├── shared/
├── database/
├── middlewares/
├── routes/
├── utils/
├── types/
└── server.ts
```

5.1 modules/ — Domain-Driven

Each module corresponds to a business domain and, where applicable, aligns with an MFE on the frontend. All module files follow a consistent naming pattern.

modules/user/

```
modules/
└── user/
    ├── user.controller.ts   # HTTP request/response handling
    ├── user.service.ts      # Business logic
    ├── user.repository.ts   # Database access layer
    ├── user.routes.ts       # Module route definitions
    ├── user.schema.ts       # Validation schemas (Zod)
    └── user.types.ts        # Module TypeScript types
```

File	Responsibility
*.controller.ts	Handles HTTP req/res; delegates to service
*.service.ts	Business logic; orchestrates repository calls
*.repository.ts	Database queries; abstracts Prisma/MongoDB
*.routes.ts	Defines and exports Express Router for this module
*.schema.ts	Zod validation schemas for request validation
*.types.ts	TypeScript interfaces and types for the module

Module Benefits

- Clean separation of concerns within each domain
- Independent scaling — modules can be split into microservices

- Clear team ownership boundaries
- Consistent file naming across all modules

5.2 shared/

Reusable backend logic shared across all modules. Keep this lean — only truly cross-cutting concerns belong here.

```
shared/
shared/
├── logger/         # Structured logging (Winston, Pino)
├── errors/         # Custom error classes & handlers
└── response.ts     # Standardized API response helpers
```

5.3 database/

Database configuration, Prisma schema, and migration files. The index.ts exports a singleton database connection.

```
database/
database/
├── prisma/
│   └── schema.prisma   # Prisma data model
├── migrations/         # Database migration files
└── index.ts            # DB connection singleton
```

5.4 middlewares/

Express middleware functions applied globally or to specific route groups. Keep middleware pure — no business logic.

```
middlewares/
middlewares/
├── auth.middleware.ts    # JWT validation, session check
└── error.middleware.ts   # Global error handling
```

5.5 routes/

Central route registration file that assembles all module routers. This is the single source of truth for the API surface.

```
routes/index.ts
routes/
└── index.ts    # Imports and registers all module routes

// Example:
```

```
import { userRouter } from "../modules/user/user.routes"

router.use("/users", userRouter)
```

5.6 utils/

Pure backend utility functions with no side effects. Safe to import from any layer.

```
utils/
utils/
├── hash.ts     # Password hashing (bcrypt)
└── token.ts    # JWT generation and verification
```

5.7 server.ts

The application entry point. Responsible for bootstrapping Express, registering middleware, mounting routes, and starting the HTTP server.

```
server.ts
server.ts — Responsibilities

1. Initialize Express application
2. Register global middleware (cors, helmet, body-parser)
3. Mount route index (/api/v1)
4. Register error handling middleware
5. Connect to database
6. Start HTTP server on configured port
```

06 Architecture Principles

These guiding principles apply across all layers of the stack. Following these ensures the codebase remains maintainable, scalable, and team-friendly as the product grows.

Principle	Description	Enforced By
Feature Encapsulation	Features own their logic, UI, and data access	features/ boundary
Public API via index.ts	Internal impl details stay private	Barrel exports
No Cross-Module Imports	Modules never directly import from each other	Code review, ESLint
Pages = Composition Only	Pages compose features; no business logic	Architecture review
Shared = Dumb	Shared components have no feature dependency	Code review
Domain-Aligned Backend	Backend modules mirror frontend domains	Module structure

6.1 Dependency Direction

Dependencies should always flow in one direction: outward from the core domain, never inward from UI.

```
Dependency Flow
pages/  →  features/  →  services/  →  shared/
                  ↓
              utils/ types/

❌ shared/ → features/  (forbidden)
❌ utils/  → features/  (forbidden)
❌ Module A → Module B  (forbidden)
```

6.2 MFE Readiness Checklist

Before extracting a feature into a standalone MFE, verify each item:

- Feature is self-contained in features/<name>/ with its own components, hooks, services, types
- Public API is cleanly defined in index.ts
- No cross-feature direct imports (only via index.ts)
- No shared global state dependencies that would break isolation
- Backend module aligned and independently deployable
- Communication contracts (props/events) are documented

6.3 Code Review Checklist

- ✅ Folder and file naming follows conventions

- ✅ No business logic in pages/ or shared/
- ✅ Features expose only via index.ts
- ✅ No cross-module imports
- ✅ Store files end in .store.ts, schemas in .schema.ts
- ❌ Business logic in page components
- ❌ Direct database queries in controllers
- ❌ Shared components importing from features/

07 Quick Reference

Use this section as a daily reference for where files belong and how they should be named.

Where does my file go?

What are you creating?	Where it lives	Naming
Reusable button/input	src/shared/components/	PascalCase.tsx
Feature business logic	src/features/<name>/	camelCase.ts
Page component	src/pages/	PascalCase.tsx
Auth hook (global)	src/hooks/	useAuth.ts
Feature-specific hook	src/features/<name>/hooks/	useFeature.ts
Global API config	src/services/	httpClient.ts
Global state	src/stores/	auth.store.ts
Global validation	src/validators/	email.schema.ts
Layout wrapper	src/layouts/	DashboardLayout.tsx
Backend API endpoint	src/modules/<name>/	*.controller.ts
Backend business logic	src/modules/<name>/	*.service.ts
Database query	src/modules/<name>/	*.repository.ts

Common Mistakes to Avoid

✗ Anti-patterns

- Importing directly into a feature from another feature (use index.ts)
- Adding API calls to page components (use services/ or feature hooks)
- Putting business logic in shared/ components
- Using a module folder name in PascalCase (use kebab-case)
- Skipping index.ts in a feature (breaks MFE extraction)
- Writing DB queries in controllers (use repository pattern)
- Cross-module imports in the module frontend

End of Developer Handbook v1.0